PRAKTIKUM 1
Membuat program dengan Visual C++ dan Python

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Riyanto Sigit, ST, M.Kom, Ph.D
MATA KULIAH	: Praktikum Sistem Cerdas
TANGGAL	: Rabu, 19 Februari 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 1

Membuat Program dengan Visual C++ dan Python

A. Tujuan

1. Mahasiswa dapat melakukan instalasi MS Visual C++
2. Mahasiswa dapat melakukan instalasi PyCharm
3. Mahasiswa dapat membuat aplikasi dengan MS Visual C++ dan

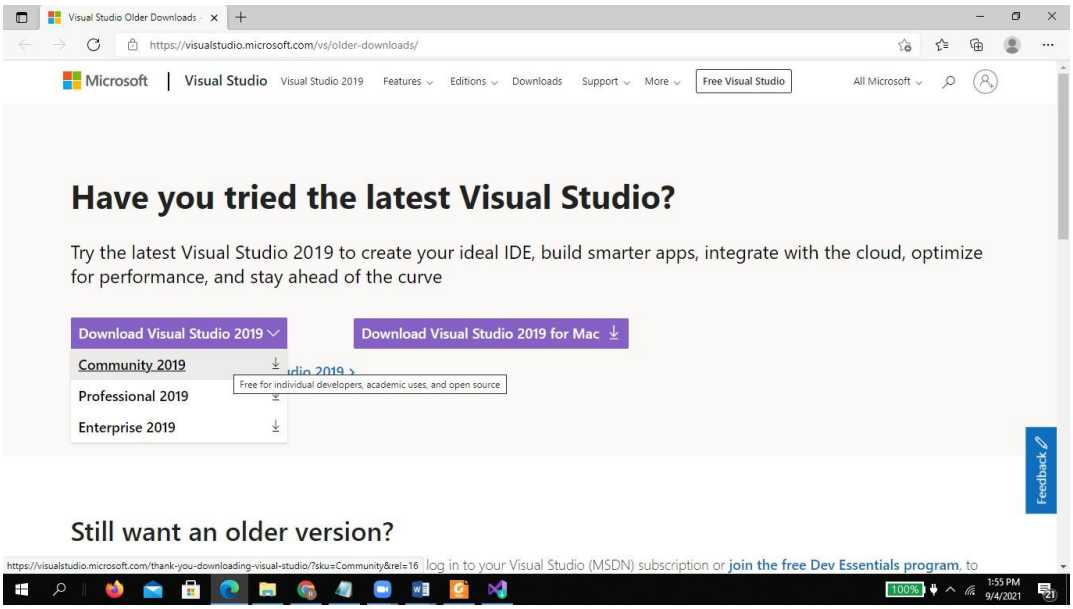
PyCharm Software yang diperlukan:

- Microsoft Visual C++
- PyCharm

B. Percobaan

1. Instalasi MS Visual C++

Masukkan CD MS Visual C++ (pada buku ini digunakan MS Visual C++ 2010) untuk instalasi, atau kunjungi website berikut dan ikuti Microsoft Guide untuk menginstal Visual C++. Visual Studio Older Downloads - 2017, 2015 & Previous Versions (microsoft.com)/

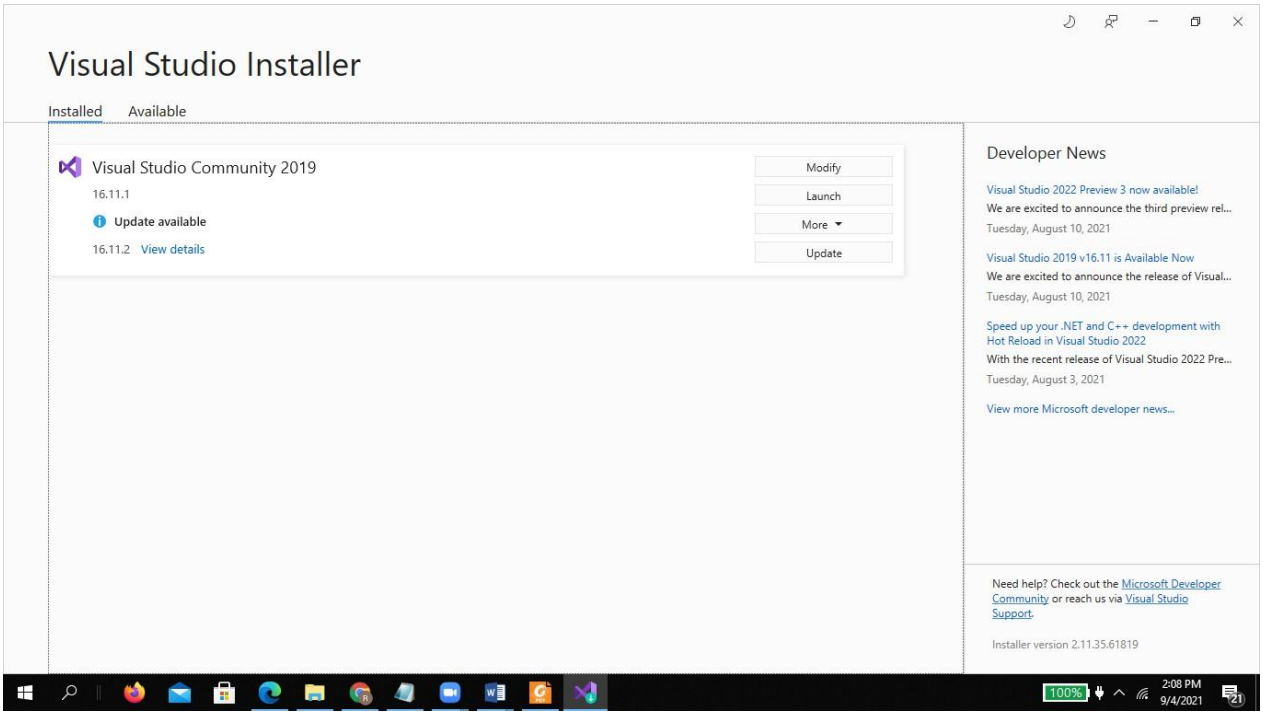


Gambar 1. Download Visual C++

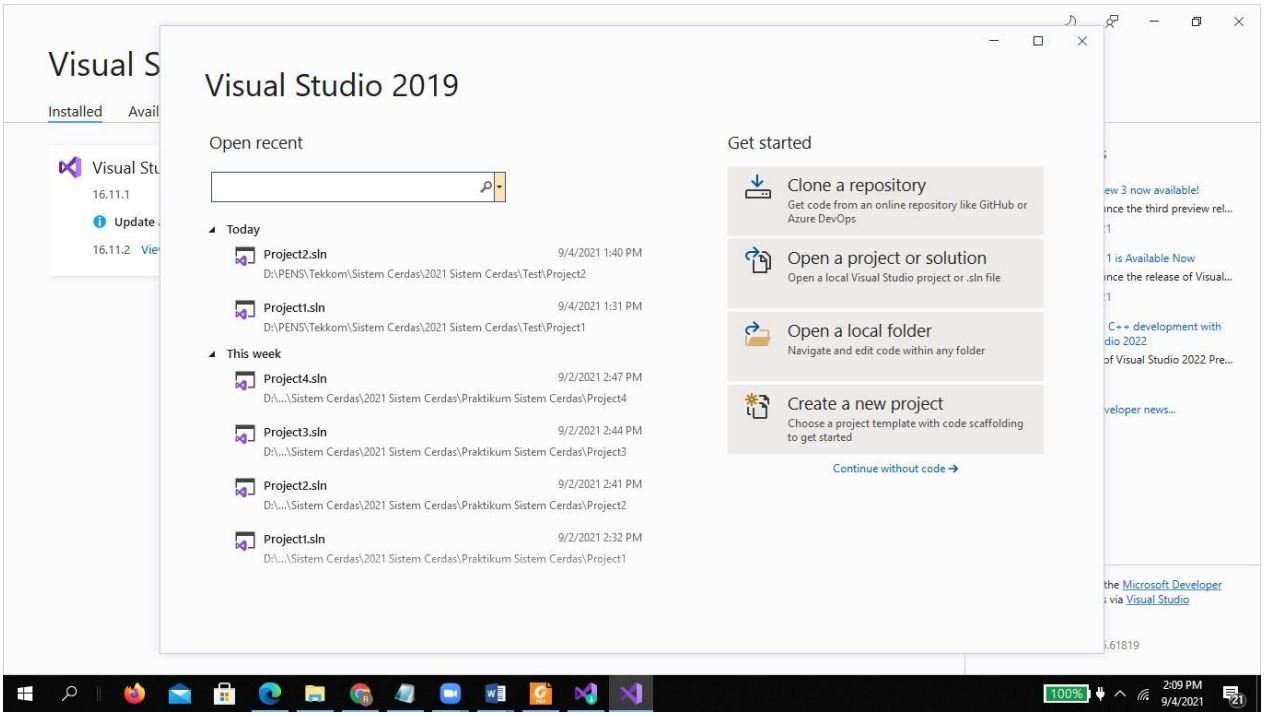
Pilih Download Visual Studi 2019 (untuk window)

Pilih Community 2019

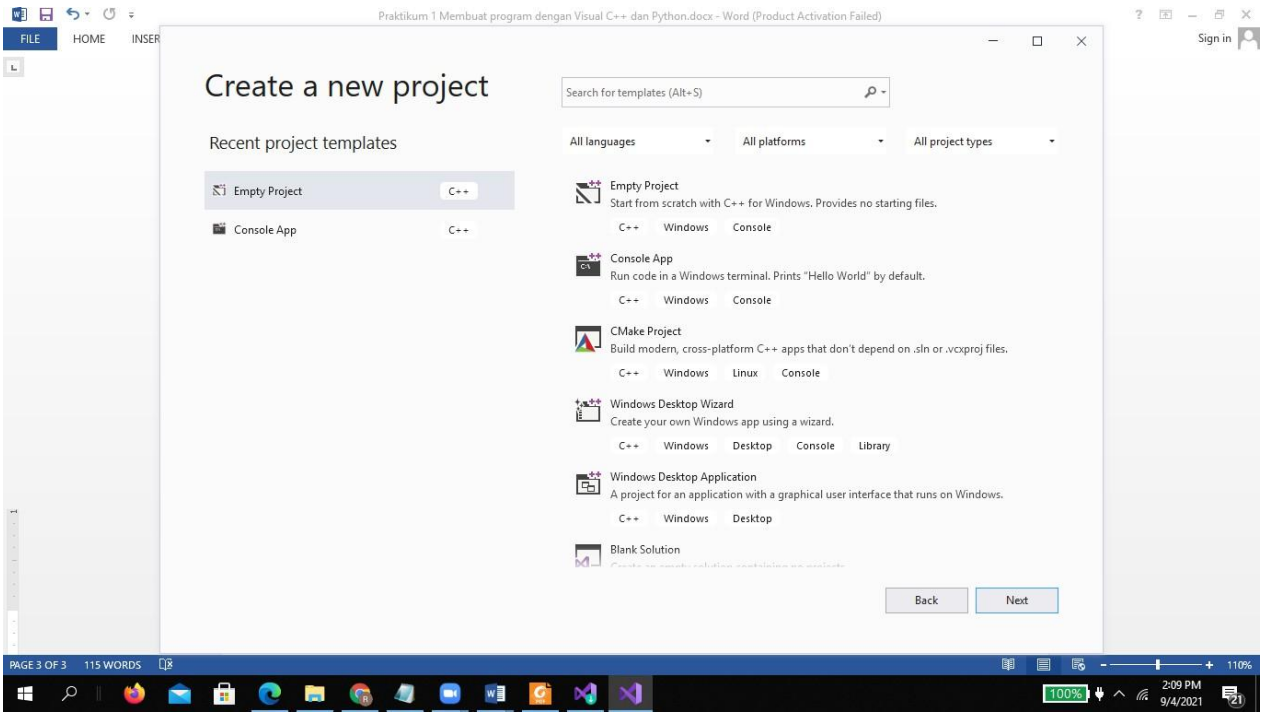
2. Run MS Visual C++



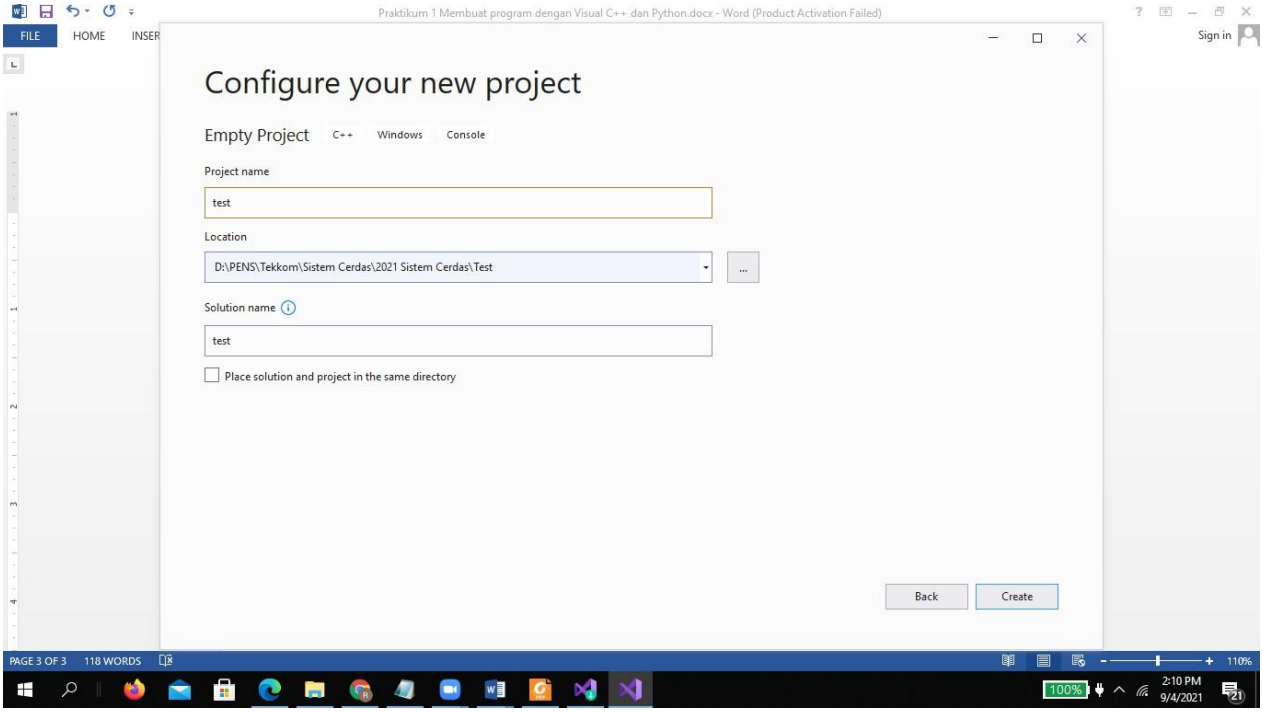
Pilih Launch



Pilih Create a new project

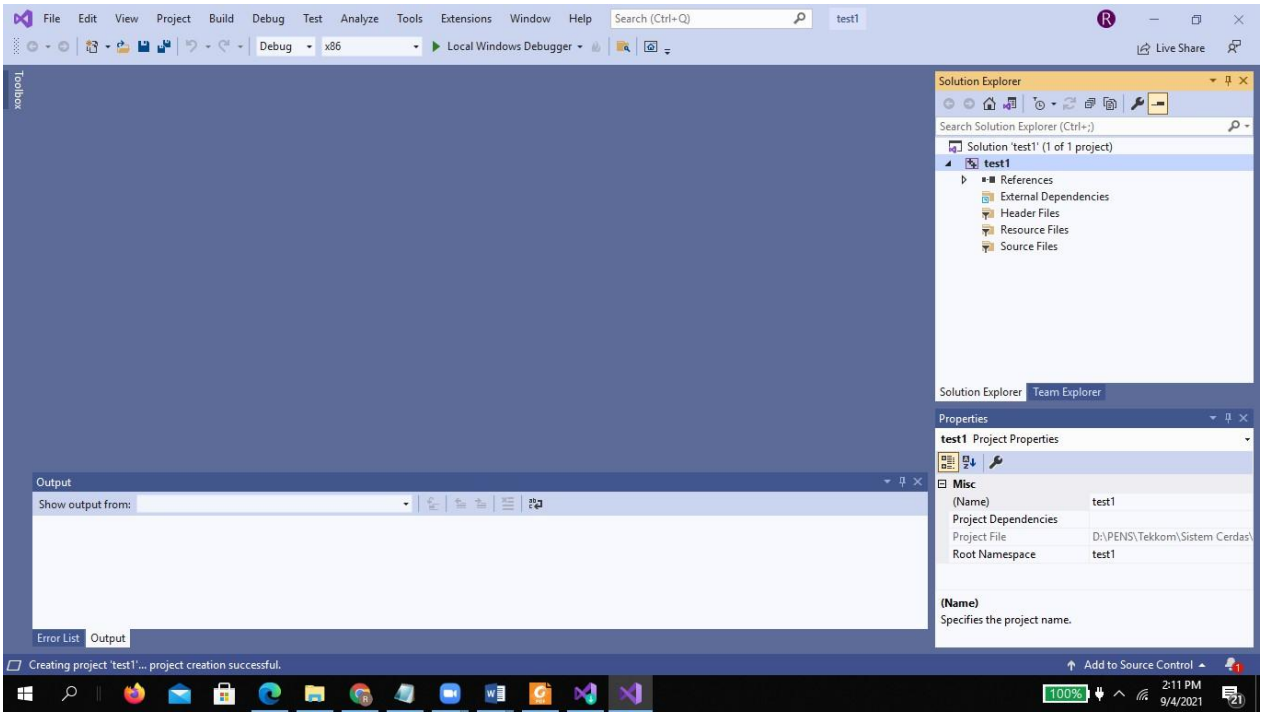


Pilih Empty Project

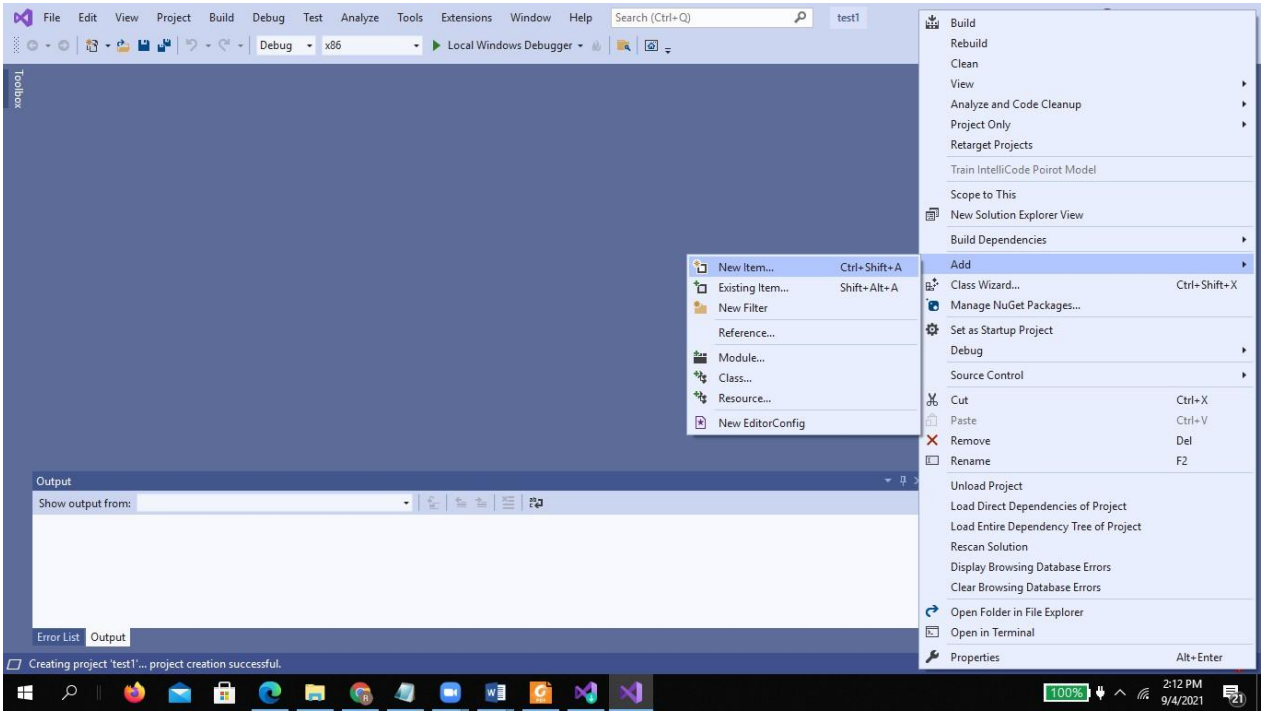


Isi project name dengan misalnya: test1

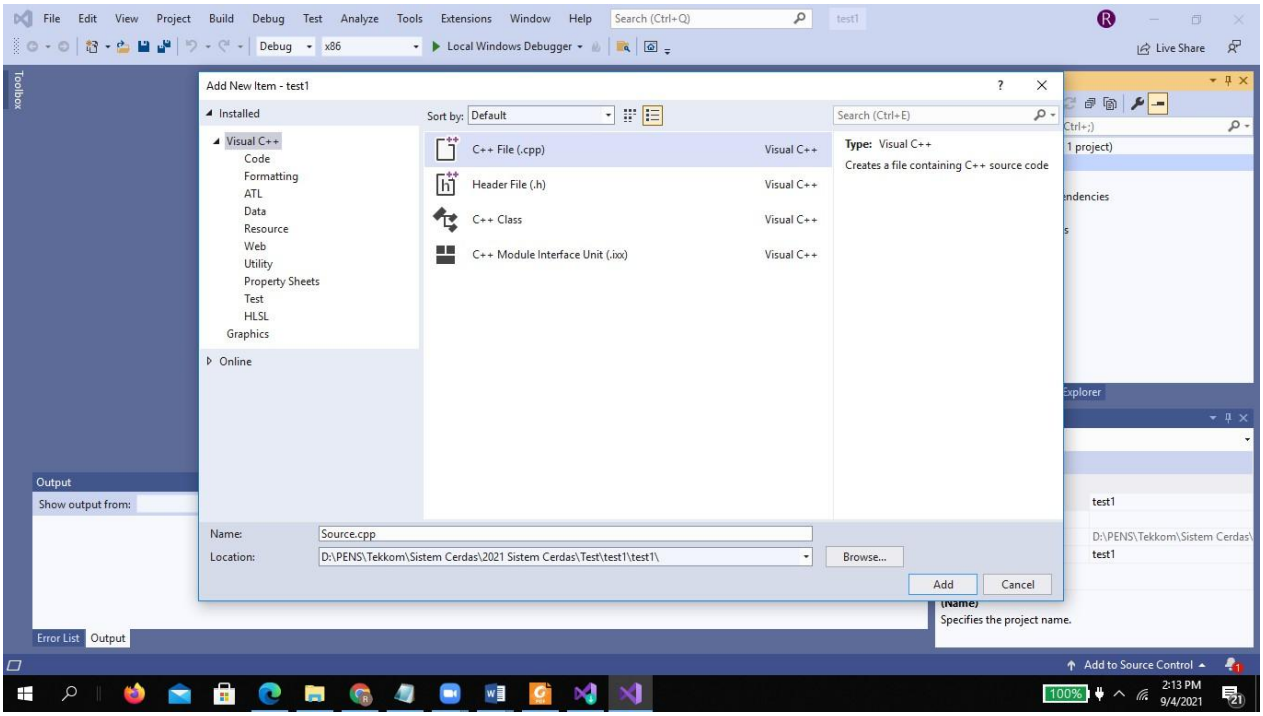
Pilih Create



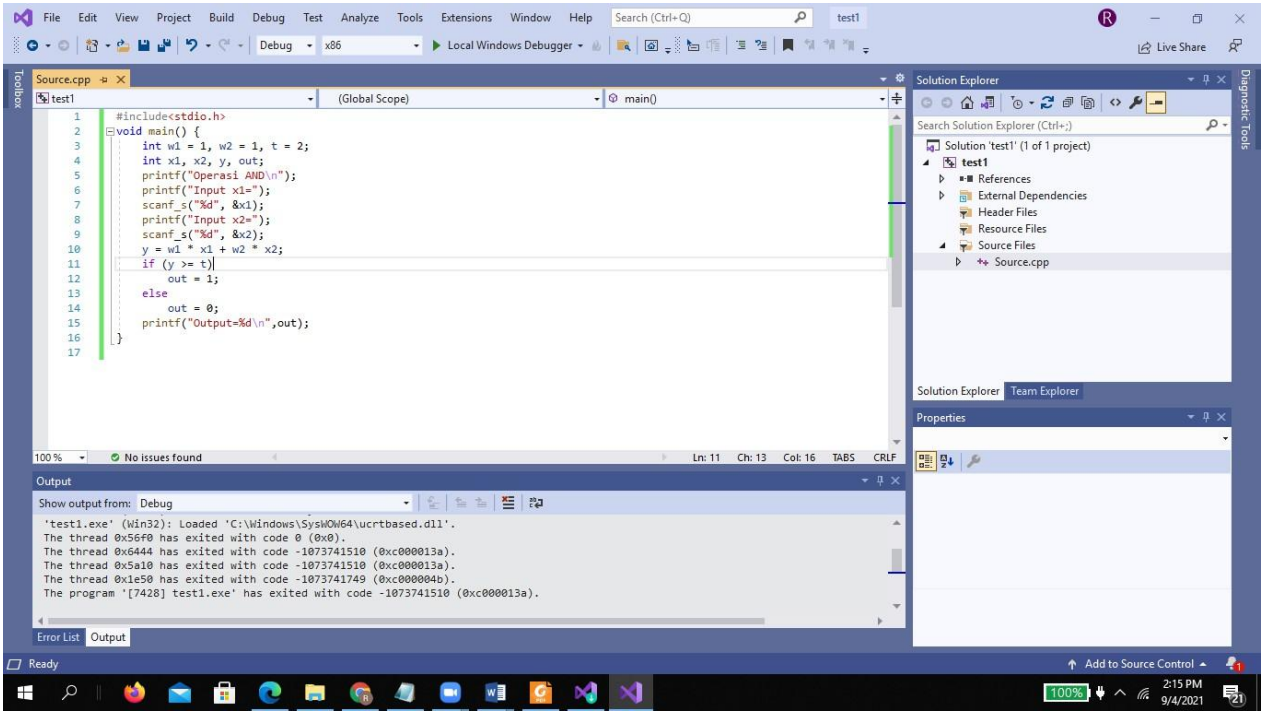
Klik kanan pada test1



Pilih Add->New Item



Pilih C++ File(.cpp)

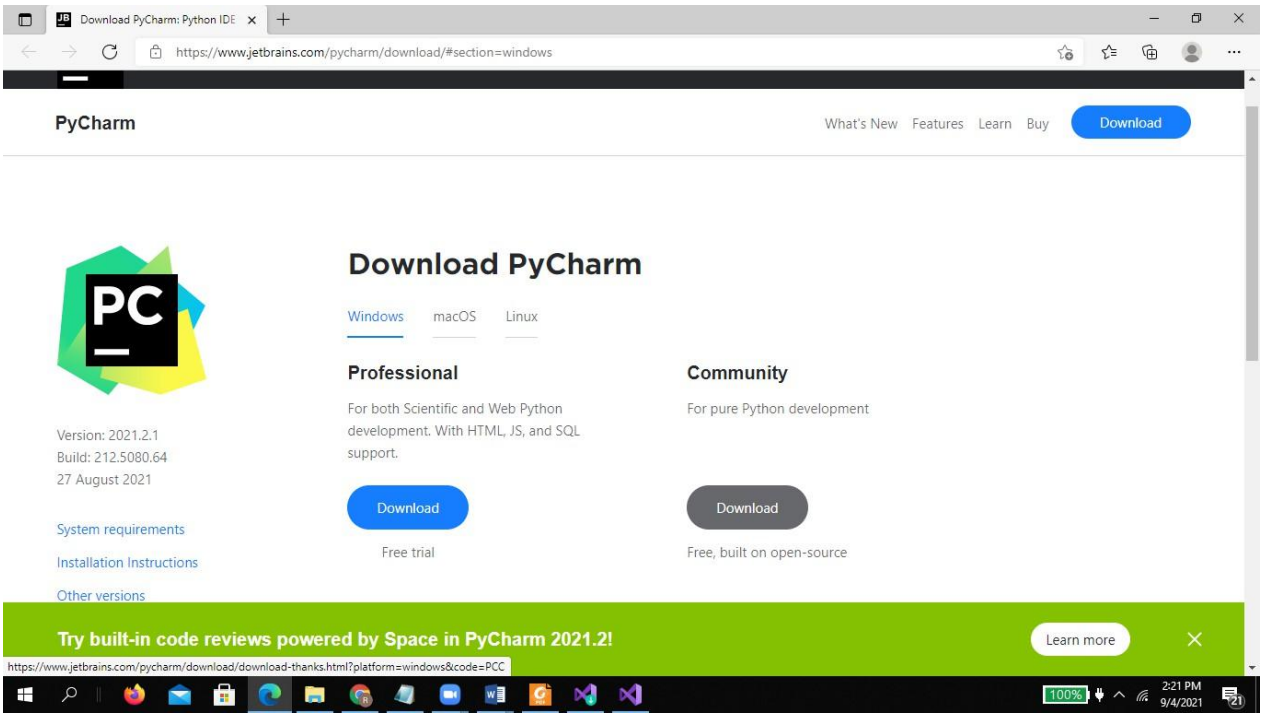


Buat program seperti diatas

Untuk Run -- Pilih Local Windows Debugger

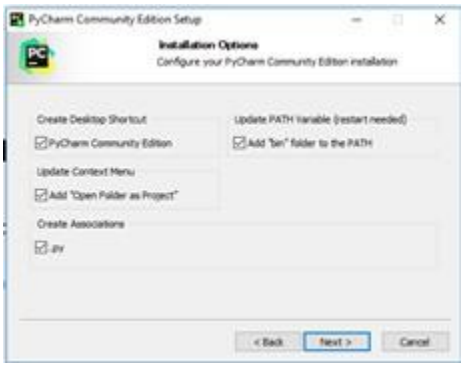
3. Instalasi PyCharm di windows

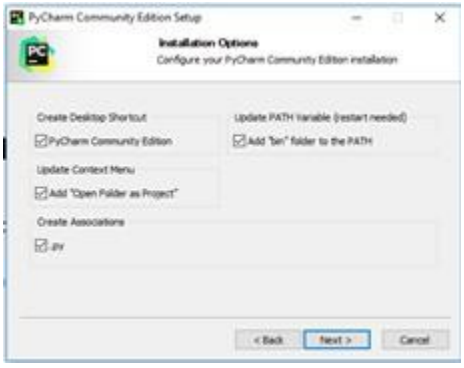
Download installer .exe in https://www.jetbrains.com/pycharm/download/#section=windows. Pilih versi Community. Ukuran file sekitar 366 MB.



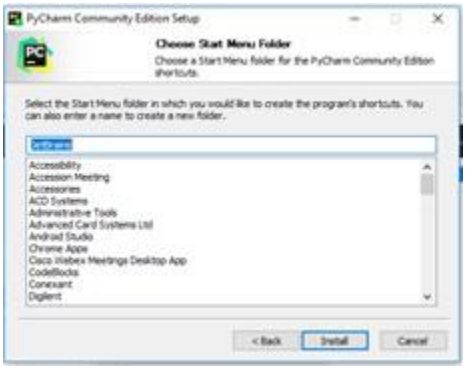
Pilih Community (tombol Download)

Pilih Next





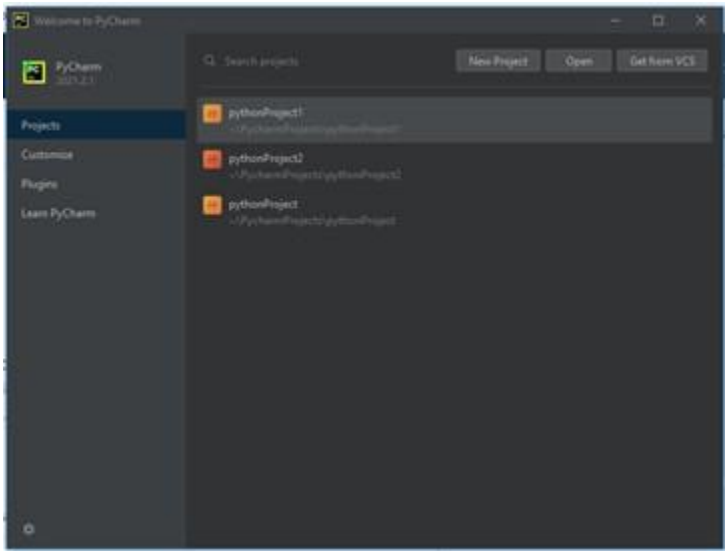
Pilih semua dan Next



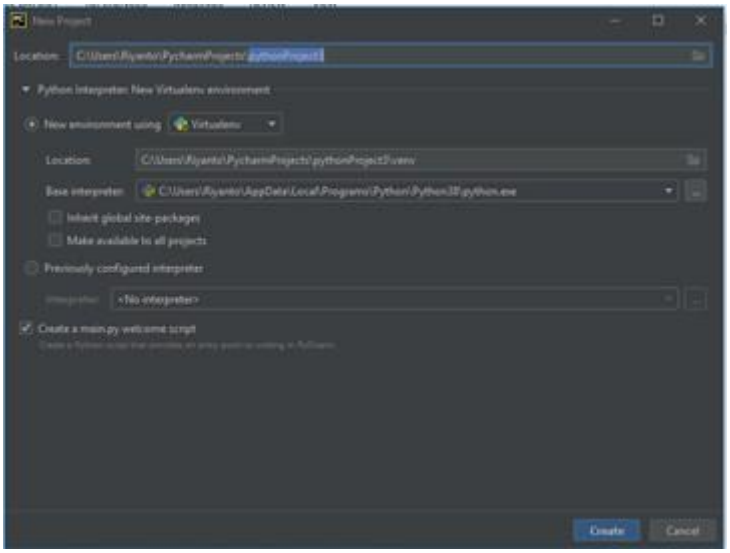
Pilih Install

Pilih Finish

Run PyCharm

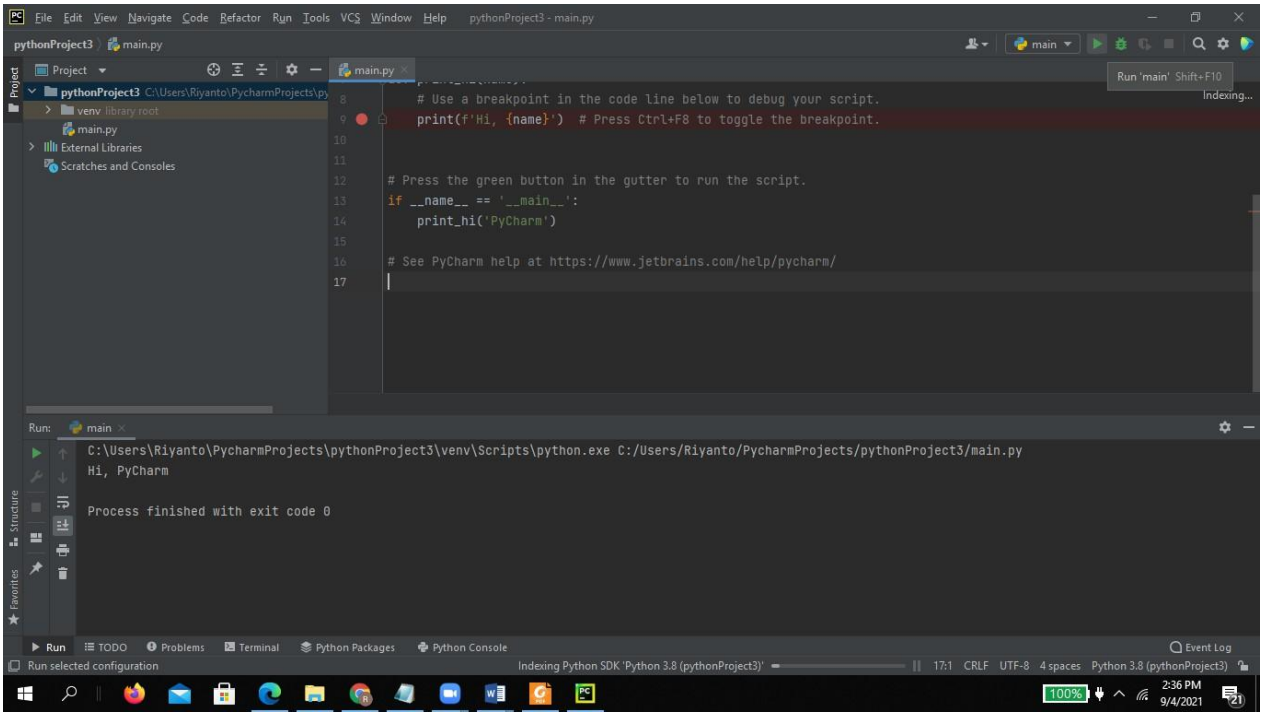


Pilih New Project

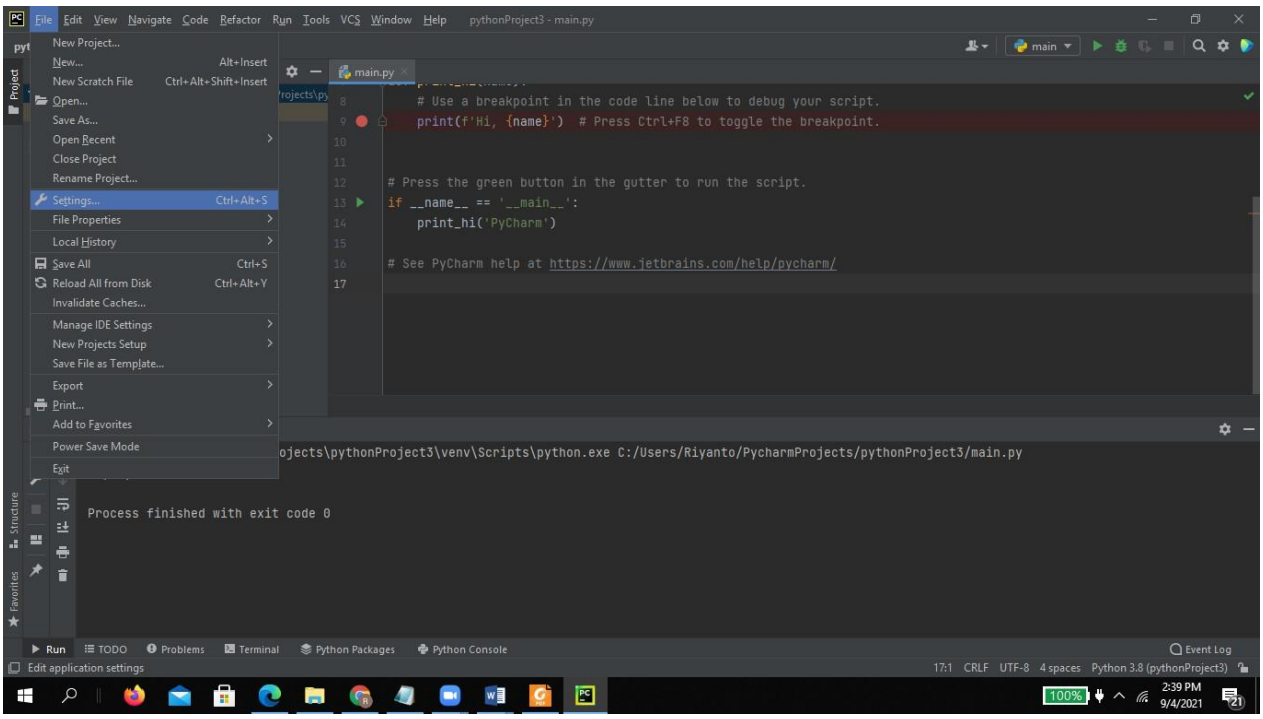


Pilih Create

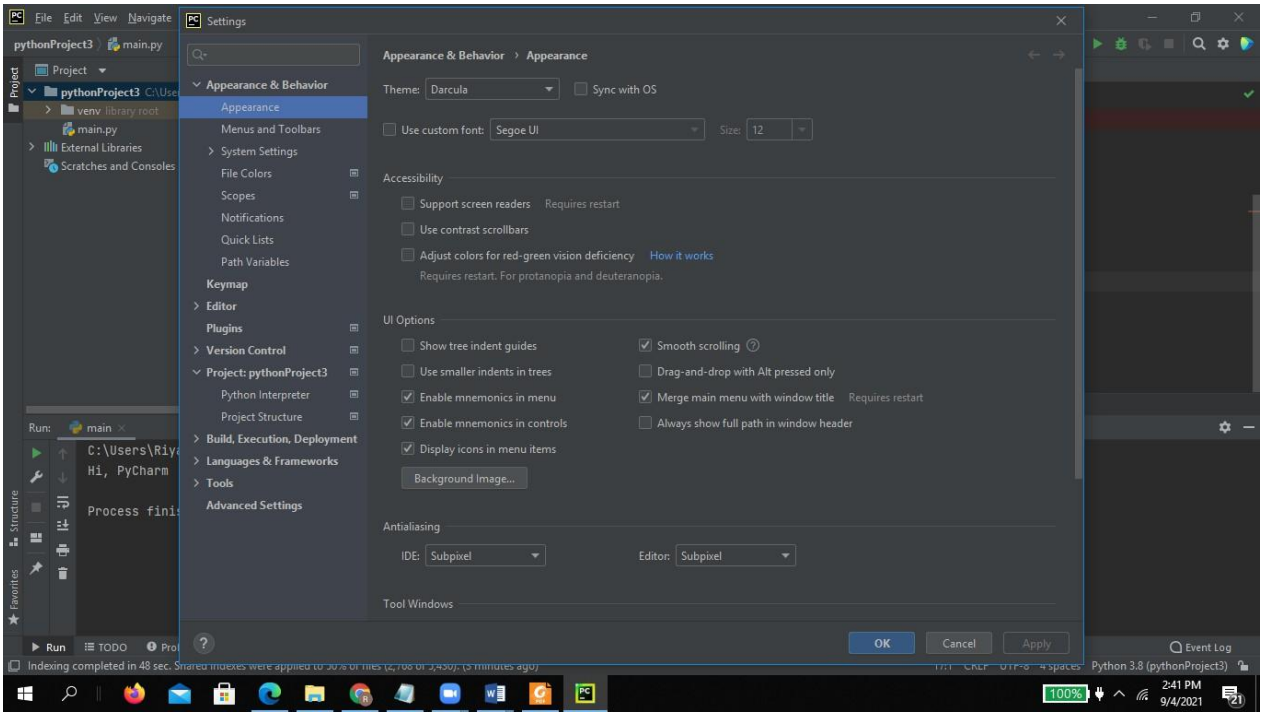
Pilih Run ‘main Shift + F10



Setting



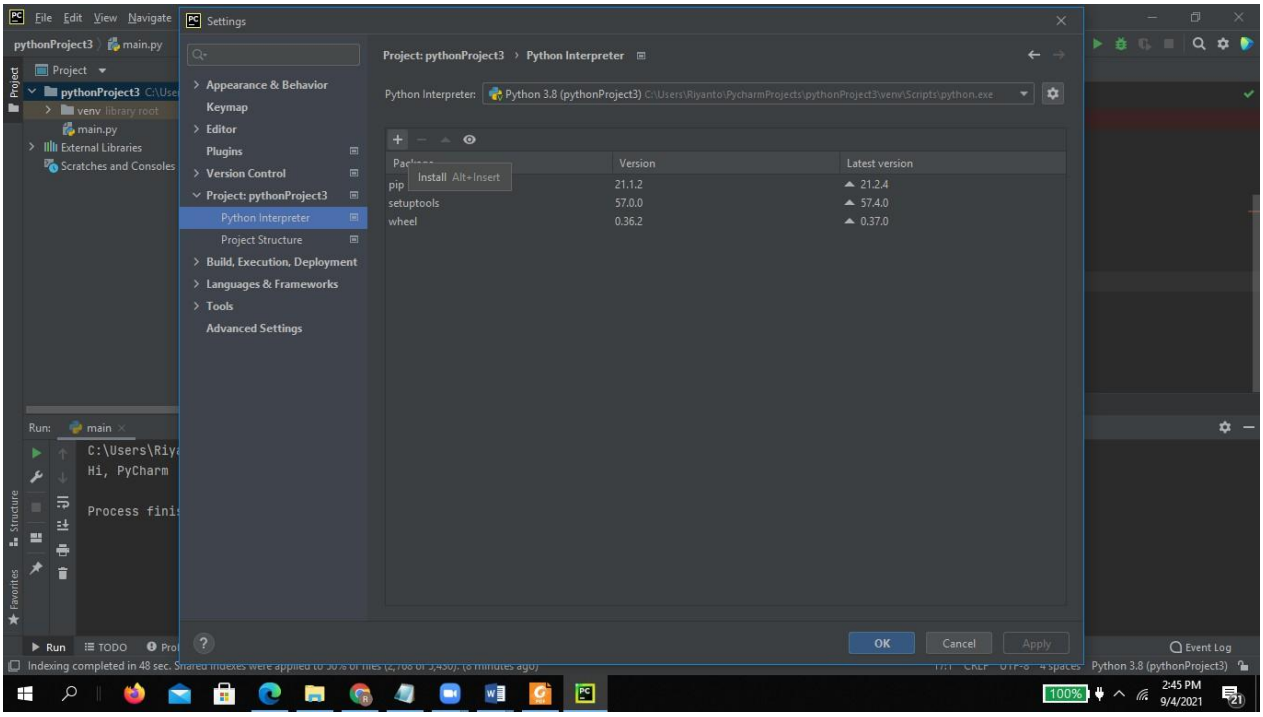
Pilih File Setting



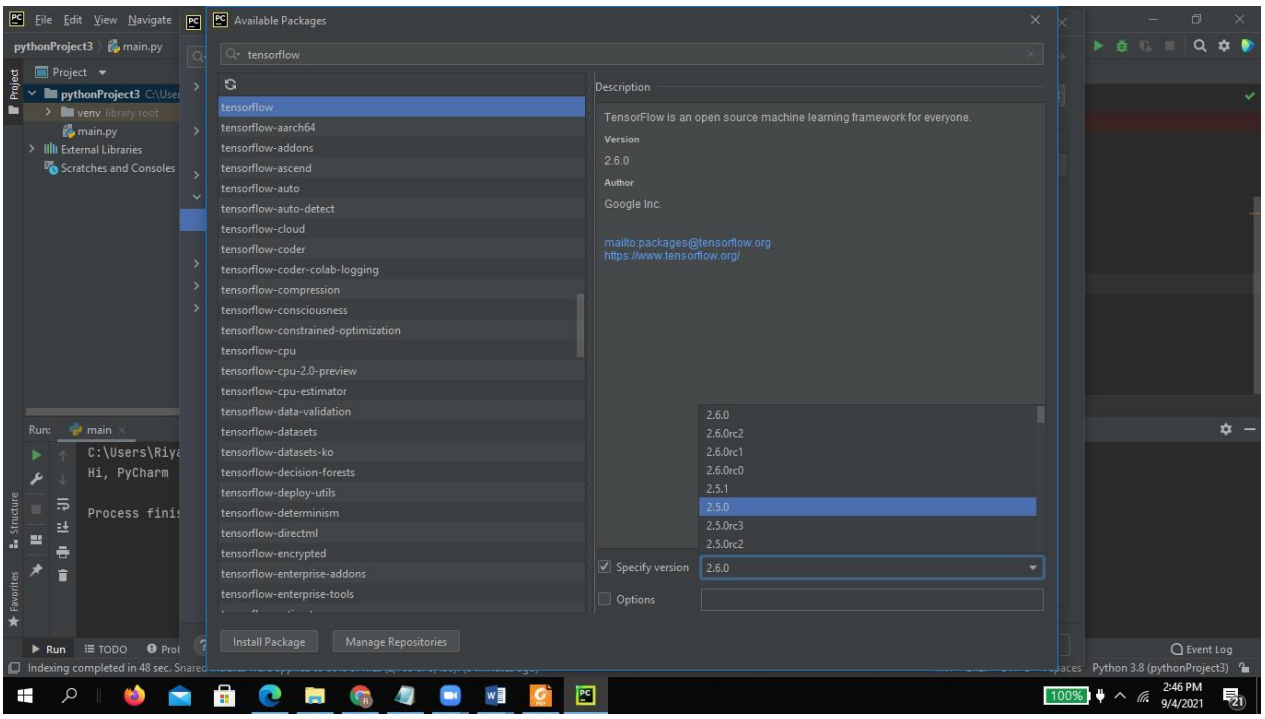
Pilih Project -> Python interpreter

Install library dibawah ini

- tensorflow==2.5.0
- tensorflow-datasets==4.3.0
- Pillow==8.2.0
- pandas==1.2.4
- numpy==1.19.5
- scipy=1.7.0

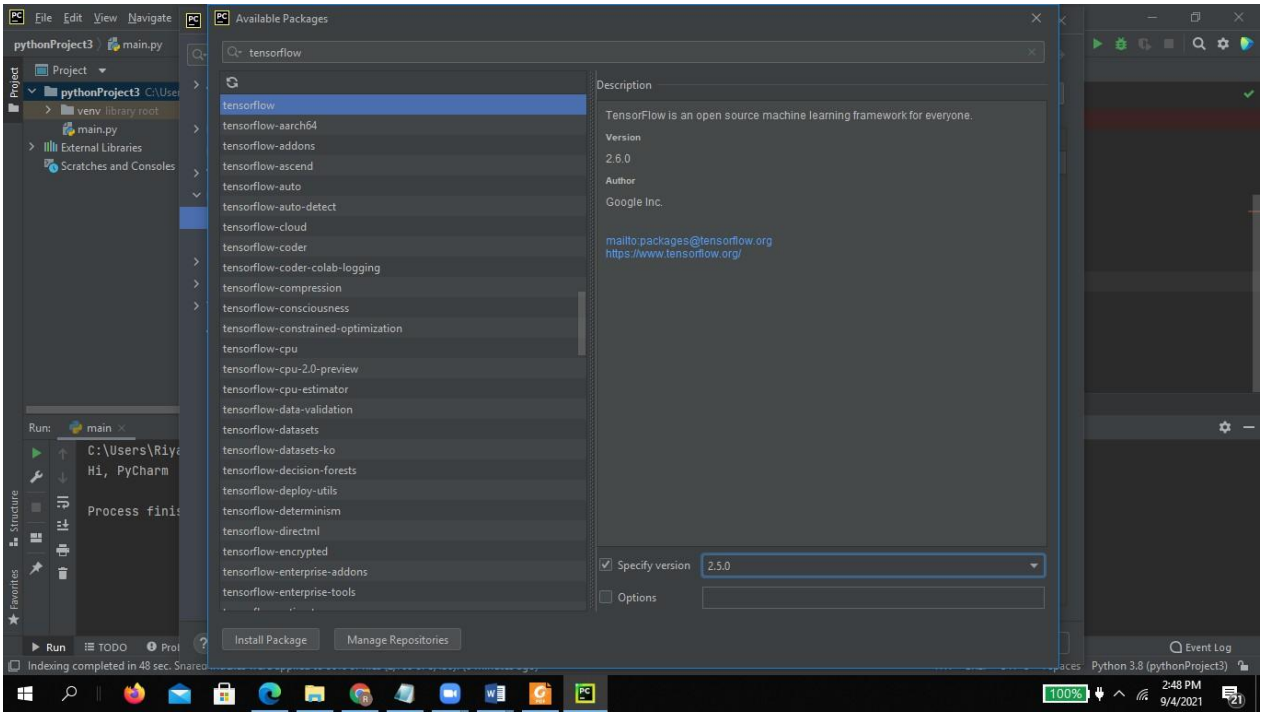


Pilih + (Install)

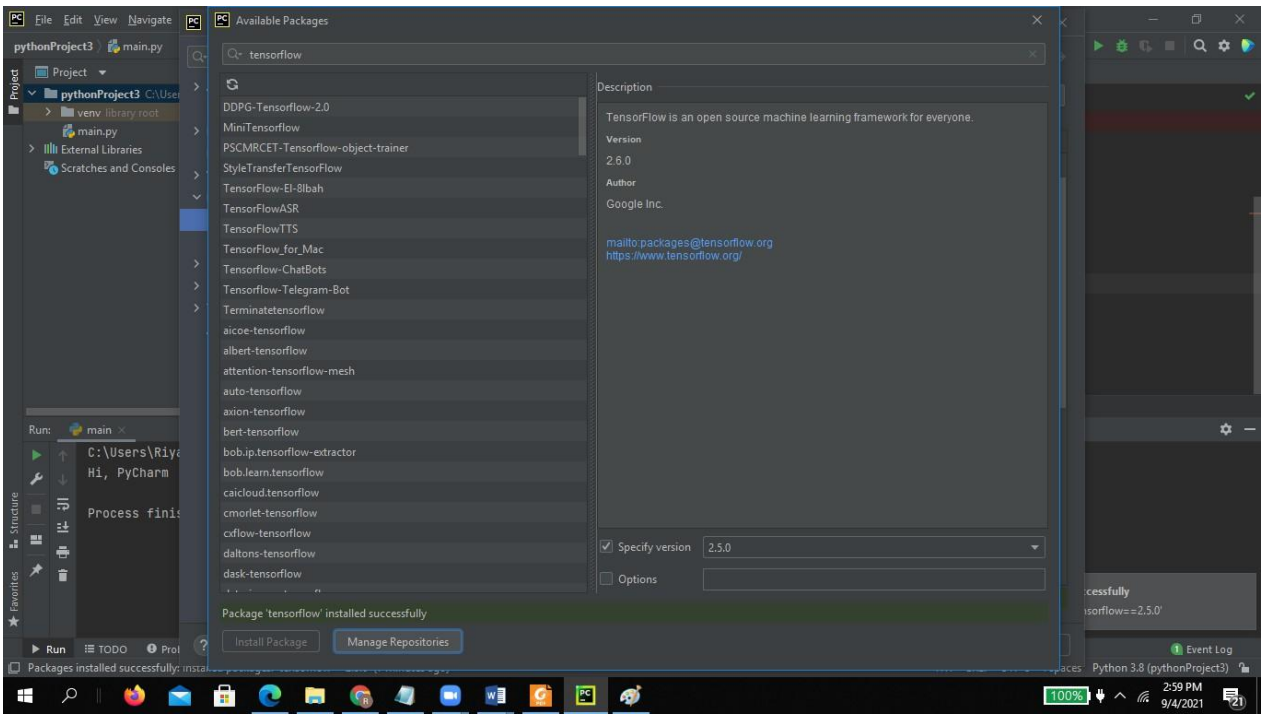


Cari tensorflow 2.5.0

Pilih Specify version dan pilih 2.5.0

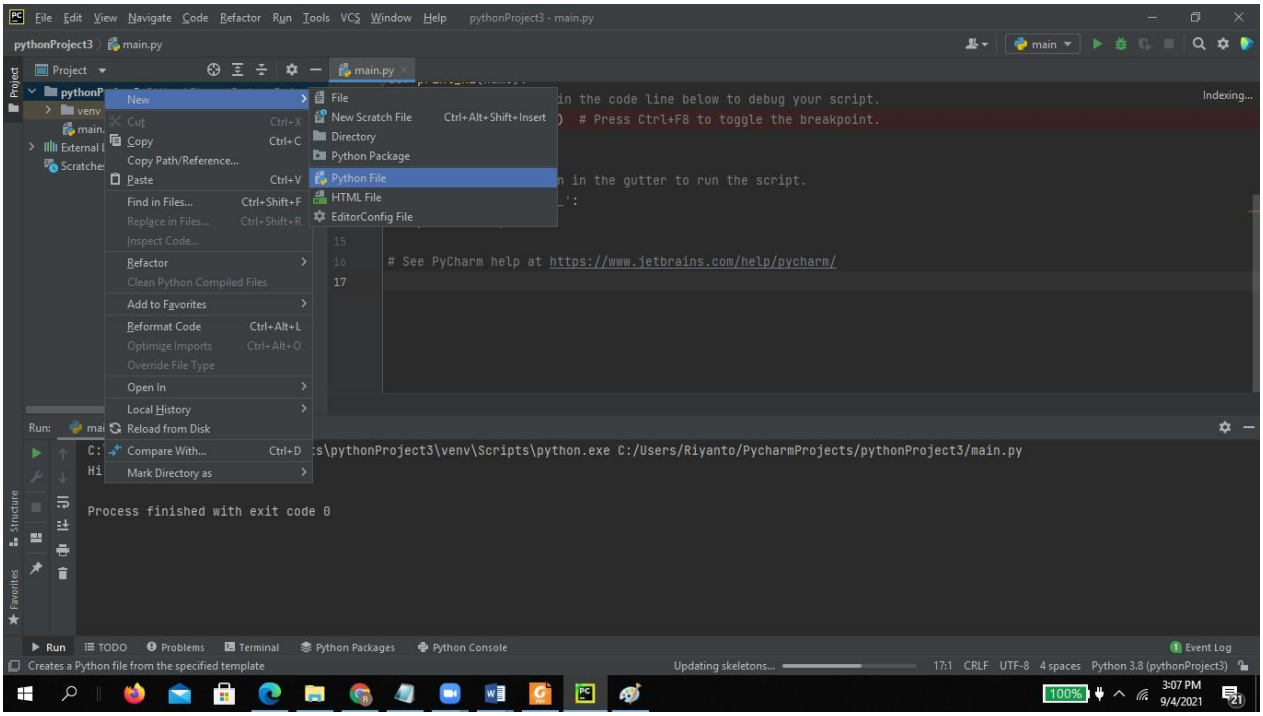


Install Package

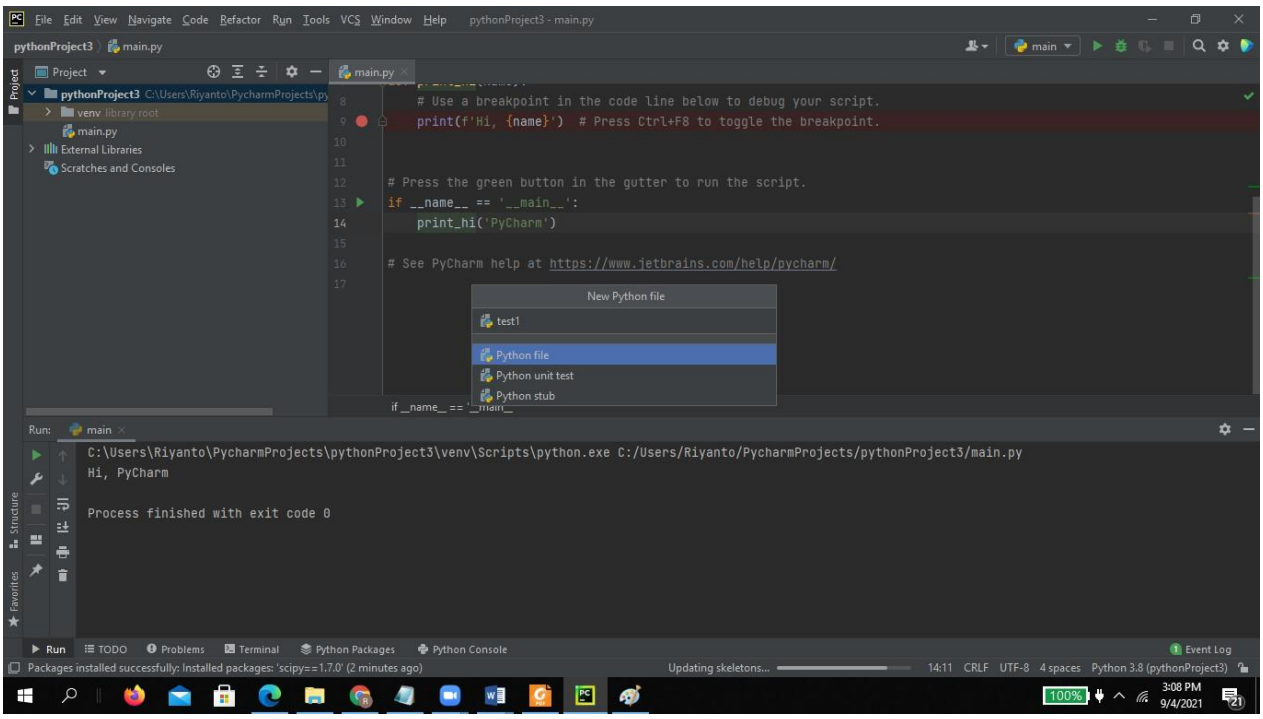


Install sukses

Run program Python



Pilih New -- Python File



Isi test1

Contoh program : **simple_regression.py**

```python
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow import keras


def plot_data(x_data, y_data):
    fig, ax = plt.subplots()

    ax.plot(x_data, y_data, 'ro')

    plt.pause(1)

    return None


if __name__== "__main__":
    # Define and compile the neural networks
    model = tf.keras.Sequential(
        [keras.layers.Dense(units=1, input_shape=[1])])
    model.compile(optimizer="sgd", loss="mean_squared_error")

    # Providing the data
    xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0])
    ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])  # y = 2x - 1

    # Plotting data will help us to understand faster
    # visually about the data
    plot_data(xs, ys)

    # Train the neural network
    model.fit(xs, ys, epochs=500)

    # Predict using the model
    print(model.predict(([10.0])))   # this result should be close to 19.
```

C. Hasil Percobaan

1. Program AND pada Visual Studio 2022

```c
#include <stdio.h>
void main() {
    int w1 = 1, w2 = 1,t = 2;
    int x1, x2, y, out;
    printf("Operasi AND\n");
    printf("Input x1=");
    scanf_s("%d", &x1);
    printf("Input x2=");
    scanf_s("%d", &x2);
    y = w1 * x1 + w2 * x2;
    if (y >= t)
        out = 1;
    else
        out = 0;
    printf("Output=%d\n", out)
}
```

Hasil:

a. Input x1 = 0 dan x2 = 0

```
Operasi AND
Input x1=0
Input x2=0
Output=0
```

b. Input x1 = 1 dan x2 = 0

```
Operasi AND
Input x1=1
Input x2=0
Output=0
```

c. Input x1 = 0 dan x2 = 1

```
Operasi AND
Input x1=0
Input x2=1
Output=0
```

d. Input x1 = 1 dan x2 = 1

```
Operasi AND
Input x1=1
Input x2=1
Output=1
```

2. Progam OR pada Visual Studio 2022

```c
#include<stdio.h>
void main() {
    int x1, x2, w1 = 2, w2 = 2, th = 2, y, out;
    printf("Operasi OR\n");
    printf("Input x1=");
    scanf_s("%d", &x1);
    printf("Input x2=");
    scanf_s("%d", &x2);
    y = x1 * w1 + x2 * w2;
    if (y >= th)
        out = 1;
    else
        out = 0;
    printf("Hasil=%d", out);
}
```

Hasil:

a. Input x1 = 0 dan x2 = 0

```
Operasi OR
Input x1=0
Input x2=0
Hasil=0
```

b. Input x1 = 1 dan x2 = 0

```
Operasi OR
Input x1=1
Input x2=0
Hasil=1
```

c. Input x1 = 0 dan x2 = 1

```
Operasi OR
Input x1=0
Input x2=1
Hasil=1
```

d. Input x1 = 1 dan x2 = 1

```
Operasi OR
Input x1=1
Input x2=1
Hasil=1
```

3. Progam XOR pada Visual Studio 2022

```c
#include<stdio.h>
void main() {
    int x1, x2, w1 = 2, w2 = -1, w3 = -1, w4 = 2, w5 = 2, w6 = 2, out1, out2, out3, th = 2, z1, z2, y, out;
    printf("Operasi XOR\n");
    printf("Input x1=");
    scanf_s("%d", &x1);
    printf("Input x2=");
    scanf_s("%d", &x2);
    z1 = x1 * w1 + x2 * w2;
    if (z1 >= th)
        out1 = 1;
    else
        out1 = 0;
    printf("Out1=%d\n", out1);
    z2 = x1 * w3 + x2 * w4;
    if (z2 >= th)
        out2 = 1;
    else
        out2 = 0;
    printf("Out2=%d\n", out2);
    y = out1 * w5 + out2 * w6;
    if (y >= th)
        out3 = 1;
    else
        out3 = 0;
    printf("Hasil=%d\n", out3);
}
```

Hasil:

a. Input x1 = 0 dan x2 = 0

```
Operasi XOR
Input x1=0
Input x2=0
Out1=0
Out2=0
Hasil=0
```

b. Input x1 = 1 dan x2 = 0

```
Operasi XOR
Input x1=1
Input x2=0
Out1=1
Out2=0
Hasil=1
```

c. Input x1 = 0 dan x2 = 1

```
Operasi XOR
Input x1=0
Input x2=1
Out1=0
Out2=1
Hasil=1
```

d. Input x1 = 1 dan x2 = 1

```
Operasi XOR
Input x1=1
Input x2=1
Out1=0
Out2=0
Hasil=0
```

4. Program simple regression pada pyCharm 2024.3.3 (Community Edition)

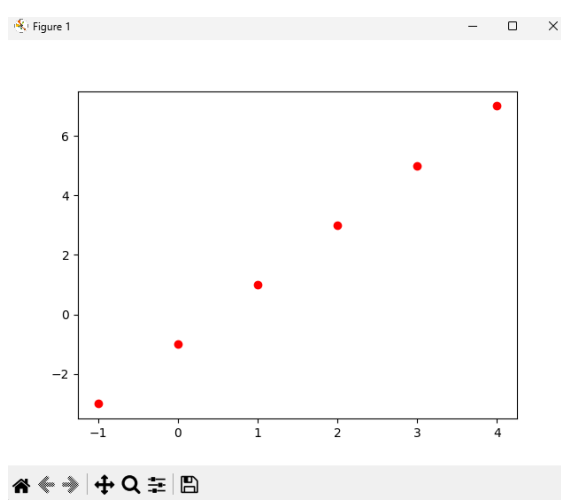
```python
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
def plot_data(x_data, y_data):  1 usage
  fig, ax = plt.subplots()
  ax.plot( *args: x_data, y_data, 'ro')
  plt.pause(1)
  return None

if __name__ == "__main__":
 # Define and compile the neural networks
 model = tf.keras.Sequential(
   [keras.layers.Dense(units=1, input_shape=[1])])
 model.compile(optimizer="sgd", loss="mean_squared_error")
 # Providing the data
 xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0])
 ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0]) # y = 2x - 1
 # Plotting data will help us to understand faster # visually about the data
 plot_data(xs, ys)
 # Train the neural network
 model.fit(xs, ys, epochs=500)
 # Predict using the model
 print(model.predict(np.array([10.0]))) # this result should be close to 19.
```

Hasil:

```
Epoch 495/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 89ms/step - loss: 2.9453e-05
Epoch 496/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - loss: 2.8849e-05
Epoch 497/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 83ms/step - loss: 2.8256e-05
Epoch 498/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - loss: 2.7675e-05
Epoch 499/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step - loss: 2.7106e-05
Epoch 500/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step - loss: 2.6550e-05
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step
[[18.98497]]

Process finished with exit code 0
```



D. Analisa

Pada praktikum ini, dilakukan implementasi gerbang logika (logic gates) AND, OR, dan XOR menggunakan bahasa pemrograman C++. Pada percobaan gerbang logika, setiap program menerima dua input biner, yaitu x1 dan x2, lalu menghitung nilai keluaran berdasarkan bobot (w) dan ambang batas (t).

1. Program AND menggunakan bobot w1=1, w2=1, serta threshold t=2. Operasi dilakukan dengan menjumlahkan hasil perkalian bobot dengan input, lalu dibandingkan dengan threshold. Jika hasilnya lebih besar atau sama dengan t, maka output adalah 1, sebaliknya 0. Hasilnya sesuai dengan tabel kebenaran gerbang AND, di mana hanya 1 AND 1 yang menghasilkan 1.

```
Operasi AND
Input x1=0
Input x2=0
Output=0
```

```
Operasi AND
Input x1=1
Input x2=0
Output=0
```

```
Operasi AND
Input x1=0
Input x2=1
Output=0
```

```
Operasi AND
Input x1=1
Input x2=1
Output=1
```

2. Program OR bekerja dengan cara serupa, tetapi menggunakan bobot w1=2 dan w2=2, dengan threshold t=2. Perbedaan utama adalah nilai bobot yang lebih besar, sehingga operasi OR menghasilkan output 1 jika setidaknya salah satu input bernilai 1. Hasil program sesuai dengan tabel kebenaran OR, di mana 0 OR 0 adalah satu-satunya kombinasi yang menghasilkan 0.

```
Operasi OR
Input x1=0
Input x2=0
Hasil=0
```

```
Operasi OR
Input x1=1
Input x2=0
Hasil=1
```

```
Operasi OR
Input x1=0
Input x2=1
Hasil=1
```

```
Operasi OR
Input x1=1
Input x2=1
Hasil=1
```

3. Program XOR lebih kompleks karena XOR bukan fungsi linear dan memerlukan dua lapisan keputusan. Program ini menggunakan dua unit tersembunyi dengan bobot berbeda untuk membentuk dua keputusan awal (out1 dan out2), yang kemudian diproses lagi untuk mendapatkan output akhir (out3). Struktur bobot dan threshold memastikan bahwa hanya kombinasi 1 XOR 0 dan 0 XOR 1 yang menghasilkan 1, sementara 0 XOR 0 dan 1 XOR 1 tetap 0. Dengan pendekatan ini, hasil yang diperoleh dari program XOR sesuai dengan tabel kebenaran XOR.

```
Operasi XOR
Input x1=0
Input x2=0
Out1=0
Out2=0
Hasil=0
```

```
Operasi XOR
Input x1=1
Input x2=0
Out1=1
Out2=0
Hasil=1
```

```
Operasi XOR
Input x1=0
Input x2=1
Out1=0
Out2=1
Hasil=1
```

```
Operasi XOR
Input x1=1
Input x2=1
Out1=0
Out2=0
Hasil=0
```

Tabel Kebenaran AND, OR, dan XOR

x1	x2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Percobaan selanjutnya adalah regresi sederhana menggunakan Python, digunakan TensorFlow untuk membuat model jaringan saraf dengan satu neuron (Dense(units=1, input_shape=[1])). Model ini dilatih menggunakan dataset xs dan ys, yang merepresentasikan hubungan linear y = 2x - 1. Data yang digunakan menggambarkan hubungan lurus pada diagram Kartesius, dan proses pelatihan menggunakan metode optimasi Stochastic Gradient Descent (sgd) untuk menyesuaikan bobot agar loss (error) sekecil mungkin. Setelah 500 epoch, loss turun hingga mendekati nol, menunjukkan bahwa model telah belajar dengan baik.

```
Epoch 495/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 89ms/step - loss: 2.9453e-05
Epoch 496/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - loss: 2.8849e-05
Epoch 497/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 83ms/step - loss: 2.8256e-05
Epoch 498/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - loss: 2.7675e-05
Epoch 499/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step - loss: 2.7106e-05
Epoch 500/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step - loss: 2.6550e-05
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step
[[18.98497]]

Process finished with exit code 0
```

Hasil prediksi untuk input 10.0 menghasilkan output sekitar 18.98, yang sangat mendekati nilai ekspektasi 19 berdasarkan persamaan linear y = 2(10) - 1 = 19. Dengan demikian, model berhasil menggeneralisasi pola dari data latih dengan akurat.

E. Kesimpulan

Program logic gate berhasil divisualisasikan menggunakan Microsoft Visual C++ dengan operasi AND, OR, dan XOR yang menghasilkan output sesuai tabel kebenaran. Proses perhitungan dilakukan dengan menjumlahkan hasil perkalian bobot dan input, lalu dibandingkan dengan ambang batas. Hasil eksekusi menunjukkan bahwa algoritma bekerja dengan benar dalam menentukan keluaran yang sesuai. Sementara itu, simple regression menggunakan Python dan TensorFlow mampu mempelajari hubungan linear y = 2x-1 dengan akurasi tinggi. Setelah dilatih selama 500 epoch, model menghasilkan prediksi yang mendekati nilai ekspektasi dengan error sangat kecil. Praktikum ini membuktikan bahwa implementasi logika dan jaringan saraf sederhana dapat dilakukan dengan baik menggunakan pendekatan pemrograman.